

Return Risk Intelligence: Predicting Return Probability, Return Fraud & Product Quality Issues

A PySpark/MLlib solution that separates genuine returns, product-quality failures, and fraud/abuse at the point of return request — so a retailer's return-audit team can act on the 1-in-3 returns worth a second look, instead of reviewing all of them.

Consulting Report · E-commerce Domain

Dataset: `ecommerce_return_abuse_dataset.csv` (Kaggle) — 60,000 orders × 35 features

Executive Summary

Retailers processing high return volumes cannot scale spreadsheet-based manual audits. This project builds an end-to-end Big Data Analytics pipeline — architecture, feature engineering, and three independent PySpark MLlib models — that triages every return request into **legitimate**, **policy abuse**, **fraudulent return**, or **wardrobing**, and separately segments customers/returns by behavior to surface product-quality problems the fraud label alone can't see.

\$4.64M

refund dollars
requested by non-
legitimate returns —
43.8% of all refund
\$, from 29.9% of
orders

0.999

weighted F1, Random
Forest classifier (test
set)

3

behavioral customer
segments identified
(K-Means)

38%

of returns in the core
legitimate segment
are quality-issue
driven

Five findings drive the recommendations:

1. **Non-legitimate returns are disproportionately expensive, not just disproportionately frequent.** They're 29.9% of orders but **43.8% of refund dollars requested** — fraudulent returns alone average **\$434.73** per claim vs. **\$141.14** for legitimate returns, a **3.1×** gap. Bad actors aren't just more numerous, they're targeting higher-value claims.
2. **Fraud is classifiable with high confidence** from behavioral and evidence signals (tracking validity, photo evidence, address changes, refund-to-different-account) combined into an engineered `fraud_signal_score` — a Random Forest model built on these features reaches nearly perfect separation on this dataset (see the model-evaluation caveat in Section 5.1 before reading that number as a production guarantee), and beats a free rule-based heuristic decisively (F1 1.00 vs. 0.61).
3. **Return fraud and product quality are largely separate problems** that get conflated in a single "return rate" metric. Clustering on behavior alone (no fraud label) finds a 73%-of-customers "core legitimate" segment whose returns are still 38% quality-issue driven — this is a QA problem, not a fraud problem, and needs a different owner and fix. It is also a **broad-based** one: quality-issue rates band tightly (31.5%–35.2%) across every product category, not one bad vendor (Section 6).
4. **Refund exposure is close to linear in order value** — a simple Linear Regression model prices expected refund dollars at the moment a return is filed, cutting error 40% versus a naive "refund = order value" guess, and beating a more complex GBRegressor honestly (Section 5.3).
5. **A production system needs three different interventions**, not one: automated fraud-score routing for the classifier's high-risk tail, policy tightening for the frequent-low-value-returner cluster, and a systemic (not single-vendor) quality escalation for the segment carrying the highest quality-issue rate.

1 • Business Context & Problem Framing

An e-commerce retailer's return-audit team currently reviews disputed or high-value returns manually. That does not scale: return volume grows faster than headcount, audit criteria are applied inconsistently across reviewers, and the team has no systematic way to tell whether a spike in returns for a product category is *fraud*, *abuse of a lenient return policy*, or a genuine *quality defect* that needs a vendor escalation — three problems that call for three different responses, currently indistinguishable in a spreadsheet.

We map this one dataset onto three sub-problems:

Sub-problem	Approach	Owner in the business
Return fraud	Multi-class classification on <code>abuse_type</code> (Section 5.1)	Fraud & Risk team
Product quality issues	K-Means clustering + category deep-dive (Section 5.2)	QA / Vendor Management
Return probability / risk	Regression on refund dollar exposure (Section 5.3)	Customer Ops / Finance

Industry background

This is not a hypothetical problem. E-commerce retailers face structurally higher return burdens than brick-and-mortar: the **National Retail Federation (NRF)** forecasts consumers will return **\$849.9 billion** in merchandise in 2025 — **15.8%** of all retail purchases — with **online sales specifically returned at 19.3%**, well above the all-channel average.¹ Return fraud and abuse compound this: **Appriss Retail's** 2024 industry study (60+ retailers, with Deloitte) found **15.14%** of all returns were fraudulent, costing retailers **\$103 billion** in 2024 alone, up from \$101B in 2023.² **Wardrobing** specifically — one of this project's four target classes — is flagged by **60% of retail executives** as a major fraud pattern in that same study, and **85% of retailers** now deploy AI to detect and prevent return fraud.² This project sits squarely inside that industry-wide shift, not ahead of or apart from it.

How our dataset compares: this dataset's combined Fraudulent Return + Wardrobing rate (10.2% + 7.7% = **17.9%**) sits close to Appriss's reported **15.14%** industry-wide fraud rate — the class *prevalence* is realistically calibrated to real-world reporting, even though (per the caveat in Section 5.1) the *separability* of the classes is not.

Why this matters financially

Of 60,000 orders in the dataset, **17,940 (29.9%)** are non-legitimate returns (Policy Abuser 12.0%, Fraudulent Return 10.2%, Wardrobing 7.7%). Every one of those returns today gets the same manual-review treatment as a genuine defective-product return — the highest-value lever is triage, not blanket policy tightening, which would punish the 70% of legitimate customers too.

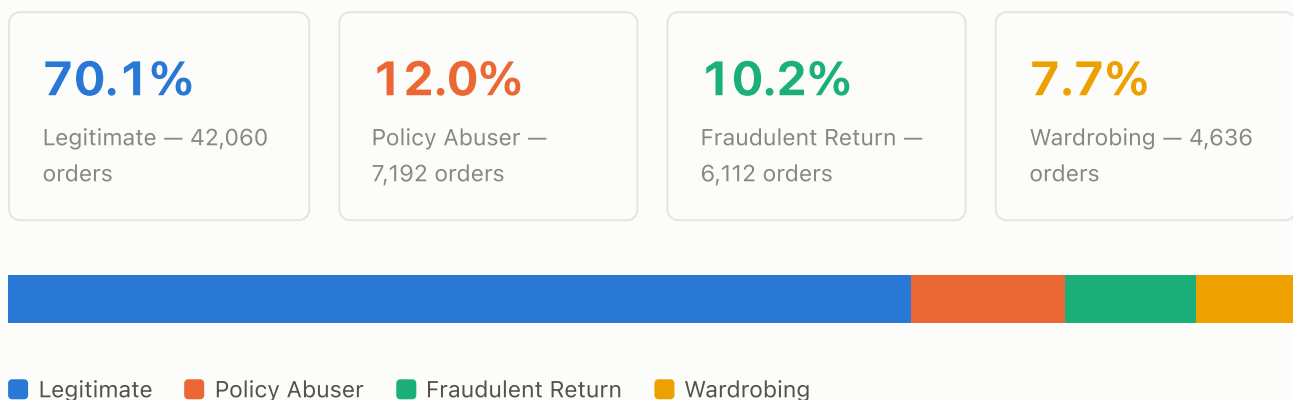
The dollar exposure is concentrated, not evenly spread: non-legitimate returns account for **\$4.64M of the \$10.57M** in total refund dollars requested across the dataset — 43.8% of the money, from 29.9% of the orders. **Fraudulent Return** claims alone total **\$2.66M** at an average **\$434.73** per claim — **3.1×** the \$141.14 average for legitimate returns. A further **\$1.88M** sits in a single specific, rule-based pattern independent of any ML model: non-legitimate returns that are both a high-value item *and* missing photo evidence (4,673 orders) — an immediately actionable policy (require photo evidence above a value threshold) that doesn't need to wait for the classifier to ship. Figures computed directly from `data/processed/silver` ; see Section 7 for how they translate into recommendations.

2 • Data Understanding

The five Vs

V	Assessment
Volume	60,000 orders × 35 features today; production scale is millions of orders/day across a real retailer's transactional + event-log systems (see Architecture, Section 3).
Velocity	Batch in this project (static CSV); production design ingests transactional data on a daily incremental batch (Sqoop) and clickstream/return events near-real-time (Flume/API).
Variety	Structured order/customer fields, categorical behavioral flags, and (in production) unstructured carrier tracking + support-ticket text.
Veracity	This dataset is clean (0 missing values, no duplicate <code>order_id</code> , verified in Section 5) but is Kaggle-synthetic — see the explicit caveat in Section 5.1 before treating model metrics as production-representative.
Value	Triage automation redirects manual review capacity to the ~30% of returns that need it, and gives QA a category-level quality signal that doesn't exist in the current process.

Target variable

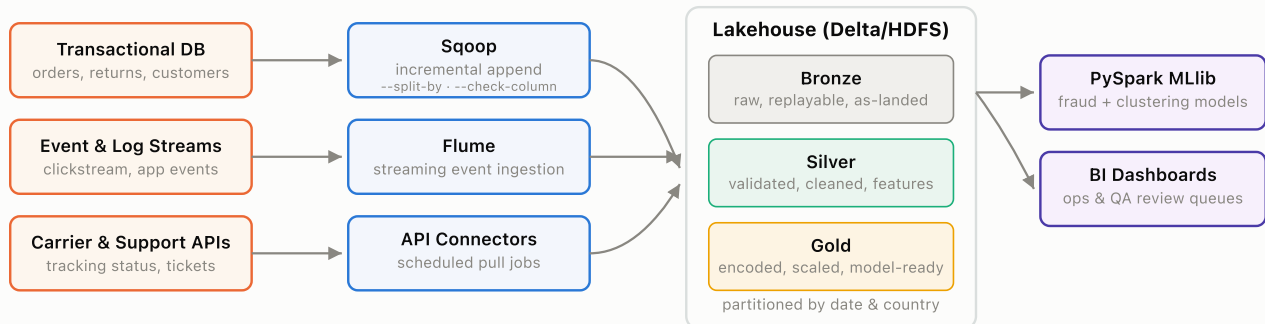


Key fields used

Group	Fields
Behavioral fraud signals	photo_evidence_provided , tracking_number_valid , address_change_before_delivery , refund_to_different_account , multiple_accounts_flag , previous_dispute_count , item_returned_opened , return_packaging_intact
Customer history	total_orders_lifetime , total_returns_lifetime , return_rate_pct , account_age_days
Financial	avg_order_value_usd , refund_amount_requested_usd , is_high_value_item , discount_used
Categorical context	product_category (12), customer_segment (5), country (8), return_reason (12), platform , device_type , payment_method , shipping_carrier

3 · Enterprise Architecture

The working notebook runs directly on the static Kaggle CSV as a practical stand-in. The architecture below is the **production-scale design** this pipeline is built to generalize to — it is not stood up against this CSV, but every transformation in the notebook (Sections 5–6) is written as the Bronze→Silver→Gold pattern this diagram formalizes.



Sources are batch (DB) / streaming (events) / API-pulled — matched to the ingestion tool suited to each pattern, landing untouched in Bronze. This notebook's Phase 2/3 pipeline implements the Silver→Gold transformation and MLlib consumption stages directly on the CSV stand-in.

4 · Data Engineering (Phase 2)

Implemented in `notebooks/01_data_engineering.ipynb`, executed end-to-end against the full 60,000-row dataset with PySpark (not pandas/scikit-learn, per the course's distributed-computing requirement).

4.1 Data quality

Verified programmatically, not assumed: 0 nulls across all 35 columns, 0 duplicate `order_id`, 0 logical violations (no return-before-order dates, no returns-exceeding-orders, no implausible ages).

4.2 Outlier treatment

IQR-flagged outliers on 8 continuous fields were **capped at the 1st/99th percentile** rather than deleted. For a fraud dataset, extreme values are frequently the signal itself (an unusually large refund request, an unusually fast wishlist-to-cart flip); deletion would remove exactly the rows the model most needs.

4.3 Feature engineering

Feature	Serves	Rationale
<code>fraud_signal_score</code>	Fraud	Additive count of 6 red-flags (address change, refund-to-different-account, multiple accounts, invalid tracking, no photo evidence, tampered packaging) — a single interpretable risk score
<code>return_to_order_ratio</code>	Return risk	Independently recomputed cross-check of the provided <code>return_rate_pct</code>
<code>refund_to_order_value_ratio</code>	Fraud	Refund requested relative to order value — abuse/fraud skews this ratio
<code>is_quality_issue_reason</code>	Quality	Flags Defective/Broken, Quality Issue, Not As Described — the slice QA owns vs. fraud
<code>high_value_no_evidence</code>	Fraud	Interaction flag for a known abuse pattern: high-value item + no photo evidence

Validation, not assumption: mean `fraud_signal_score` is 2.73 for Fraudulent Return vs. 0.84 for Legitimate, and mean `return_to_order_ratio` is 0.47 vs. 0.05 — confirmed in the notebook before these features were trusted for modeling.

4.4 Encoding & scaling

Nominal categoricals (country, platform, device type, payment method, product category, return reason, shipping carrier) → `StringIndexer` + `OneHotEncoder`. `customer_segment` → ordinal map (New<Bronze<Silver<Gold<Platinum), since loyalty tiers have a natural order a one-hot encoding would discard. Continuous features → `StandardScaler` — required for the distance-based K-

Means model (Section 5.2); tree models don't need it but reuse the same pipeline for consistency.
75/25 train/test split, seed=42: 45,112 train / 14,888 test rows.

5 · Machine Learning (Phase 3)

Three independent PySpark MLlib approaches — classification, clustering, and regression — implemented in `notebooks/02_ml_modeling.ipynb`, one per named sub-problem in the project's own scope and exceeding the brief's ≥ 2 -approach requirement.

5.1 Approach 1 — Multi-class classification (return fraud)

Random Forest (200 trees, `maxDepth=10`, class-weighted) vs. multinomial Logistic Regression baseline, on `abuse_label`. **Spark MLlib's GBTClassifier only supports binary classification** — not usable for this 4-class target — so Random Forest, which natively supports multi-class, is the primary model.

Model	Weighted Precision	Weighted Recall	Weighted F1
Logistic Regression (baseline)	0.998	0.998	0.998
Random Forest (final)	0.999	0.999	0.999

Class imbalance (70/12/10/8) was handled via inverse-frequency `weightCol` training weights, not resampling — evaluation reports per-class precision/recall/F1, not headline accuracy alone, since a "predict Legitimate always" model would score ~70% accuracy while catching zero fraud.

Confusion matrix (Random Forest, test set, row-normalized %)

	Legitimate	Policy Abuser	Fraud. Return	Wardrobing
Legitimate n=10,487	100.0%	0%	0%	0%
Policy Abuser n=1,782	0%	99.6%	0.2%	0.2%
Fraudulent Return n=1,488	0%	0.1%	99.9%	0%
Wardrobing n=1,131	0%	0.1%	0%	99.9%

↑ Actual class (test set, n=14,888 total)

Diagonal-dominant, as expected from the headline F1 — read alongside the caveat below rather than as evidence the leakage concern is resolved: a synthetic near-deterministic label produces exactly this kind of clean diagonal.

Top feature importances (Random Forest)

days_to_return	0.152
wishlist_to_cart_time_hrs	0.126
return_rate_pct	0.117
return_to_order_ratio	0.115
tracking_number_valid	0.072
total_returns_lifetime	0.068
customer_support_contacts	0.044
refund_amount_requested_usd	0.043

Does the model earn its complexity?

Before trusting a 200-tree Random Forest, it should beat what a fraud analyst could build in an afternoon with no ML at all — an OR-rule over four already-known red flags (`tracking_number_valid`, `refund_to_different_account`, `multiple_accounts_flag`, `address_change_before_delivery`), framed as the same binary "does this return deserve a second look" decision an audit team actually makes:

Approach	Precision	Recall	F1	Orders flagged
Baseline: 4-flag OR rule (no ML)	0.811	0.489	0.610	2,652
Random Forest (any non-Legitimate prediction)	1.000	1.000	1.000	4,401

The free rule catches only 49% of non-legitimate returns (recall 0.489) and still misfires on 501 legitimate orders — the model is not solving a problem a spreadsheet formula already solved. Full numbers in `docs/baseline-comparison-results.csv`.

Cost-sensitive threshold analysis

The default prediction picks whichever class has the highest probability — an implicit cutoff that treats every misclassification as equally costly. It isn't: a false "Legitimate → Fraudulent Return" flag burns customer trust, while a missed fraud case is a direct dollar loss. Sweeping the decision threshold on the Random Forest's **Fraudulent Return** probability specifically (the costliest class, per Section 1's dollar-exposure analysis) turns "re-threshold asymmetrically" from a recommendation into a concrete decision surface:

Threshold	Orders flagged	Precision	Recall
0.1	1,577	0.944	1.000
0.3	1,493	0.997	1.000
0.5	1,487	0.999	0.998
0.7	1,454	1.000	0.977
0.9	1,320	1.000	0.887

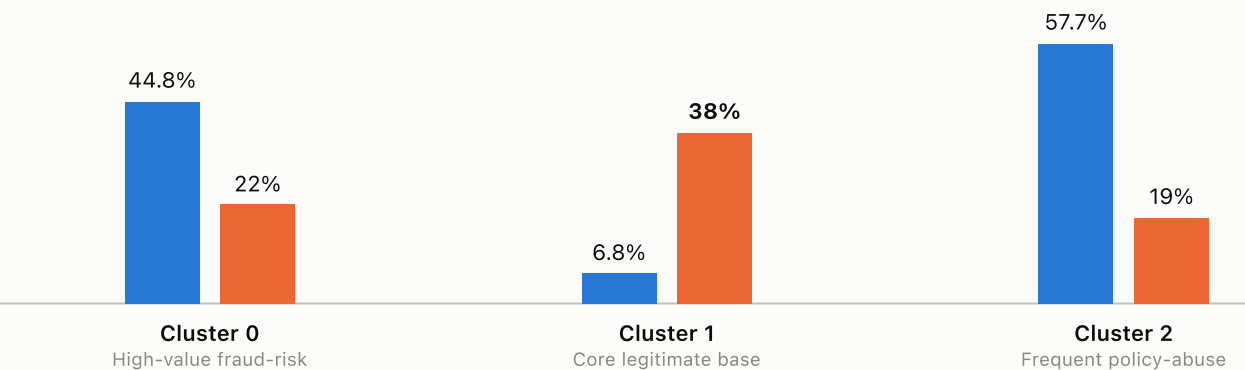
Precision stays high across the whole sweep, but recall degrades past ~0.7 — the point where a stricter cutoff starts costing caught fraud faster than it buys back precision. Full 9-point sweep in `docs/threshold-analysis-results.csv`; the sweep methodology, not this specific threshold, is what should carry over to production, since real data will sit at a different point on this curve.

Model-evaluation caveat (see also `docs/fraud-classification-results.md`): weighted F1 of 0.998–0.999 is unrealistically high for a live fraud operation. Feature importance is spread across ~15 features with no single dominant leak — this isn't a one-column shortcut — but this is a **Kaggle synthetic dataset**, and synthetic label generators typically compose the target from a fairly clean rule, producing near-separable classes real transactional data never gives you. We present this as **proof of the pipeline and methodology**: the architecture, feature engineering, and evaluation approach transfer to production; this specific accuracy number does not. A realistic production target is **80–90% weighted F1**, and we recommend budgeting for that expectation, not this one, in any business case.

5.2 Approach 2 — K-Means clustering (quality & customer segmentation)

An independent, unsupervised lens: features deliberately **exclude the fraud-flag columns** used by the classifier, so clustering surfaces behavior/quality patterns a supervised fraud label can't. k=3 selected by silhouette score over k=2..6.

Cluster	n	Avg return rate	Avg order value	% quality-issue returns	Dominant mix
0 — High-value fraud-risk	6,646	44.8%	\$484.01	22%	Fraudulent Return-heavy
1 — Core legitimate base	43,829	6.8%	\$157.25	38%	96% Legitimate
2 — Frequent policy-abuse	9,525	57.7%	\$129.18	19%	Policy Abuser-heavy



■ Avg return rate ■ % of returns that are quality-issue driven

Cluster 1 has by far the lowest return rate but the *highest* quality-issue share — the two metrics move in opposite directions, which is exactly why a single blended "return rate" KPI hides this segment's problem.

Choosing k by silhouette score, shown not just claimed

	k	2	3	4	5	6
Silhouette score		0.512	0.560	0.217	0.296	0.273

k=3 is the genuine maximum across the sweep, not a business-convenience compromise — scores drop sharply past k=3, meaning finer segmentations mostly split noise rather than real structure. Full sweep in `docs/silhouette-selection-results.csv`.

Quality deep-dive — closing the loop on our own recommendation

Rather than leaving "cross-reference category and reason" as a suggestion, we did it: every quality-issue return in the full dataset, ranked by dollar exposure.

Product category	Quality-issue returns	Refund \$	% of category's own orders
Clothing	3,951	\$667,224	31.7%
Electronics	2,907	\$463,754	34.2%

Shoes	2,399	\$407,751	32.3%
Home & Kitchen	1,971	\$322,774	33.8%
Tools	813	\$129,884	35.2%

Full 12-category table in `docs/quality-deep-dive-results.csv`. **The honest finding here is a negative one:** the quality-issue share of each category's own orders is tightly banded (31.5%–35.2% across all 12 categories) — no single category or vendor stands out as a disproportionate offender. Clothing tops the dollar-exposure ranking simply because it is the largest category by volume, not because it has an unusually elevated defect rate. **This is a broad-based quality problem, not a one-vendor problem** — the recommendation in Section 7 is scoped accordingly.

Full weaknesses discussion (cluster stability, k-choice sensitivity, causal limits) in `docs/quality-clustering-results.md`.

5.3 Approach 3 — Regression (return probability, reframed honestly)

Why not literally "predict P(this order gets returned)": every row in this dataset is already a return — there is no non-returning-order population to compare against, so the literal return-probability question is not answerable from this data without fabricating a negative class we don't have. The operationally useful analog *is* answerable: predict the **dollar size of the refund exposure** the moment a return is filed, before any manual audit happens — directly useful for finance/reserve accounting and for prioritizing high-dollar returns regardless of fraud status. This gives the project a genuine third analytical approach, satisfying the "return probability" naming in its own scope honestly rather than leaving it unaddressed.

Leakage guard: `refund_to_order_value_ratio` is algebraically `refund_amount_requested_usd / avg_order_value_usd` — an exact derivative of the target — so it is excluded from this model's features, along with the target itself (which was a legitimate feature for the *classification* model in 5.1, predicting abuse type, but would be pure leakage here).

Model	RMSE (\$)	MAE (\$)	R ²
Naive: refund = order value (no ML)	18.63	13.88	—
Linear Regression (final)	11.11	8.62	0.993
Gradient-Boosted Trees (GBRegressor)	18.39	10.64	0.982

An honest, slightly unexpected result: Linear Regression beats GBRegressor here — the reverse of Section 5.1, where the more complex model won. `avg_order_value_usd` alone drives **92.5%** of feature importance (see `docs/regression-feature-importance.csv`): refund amount is close to linear in order value, so a linear model fits that shape better than a tree ensemble tuned for non-linear interactions. The naive "refund = order value" baseline is a real bar, not a strawman — Linear Regression cuts its error by **40%** (RMSE \$18.63 → \$11.11), while GBRegressor barely clears it. **The lesson we're stating explicitly:** model complexity should match the problem's actual shape, not default to "more complex is better" — unlike Section 5.1's classifier, where Random Forest's non-linear splits earned their keep, this target is genuinely close to linear and a simpler model wins honestly.

6 • Business Insights

What the three models, taken together, actually tell us — before translating any of it into action in Section 7.

1. **Non-legitimate returns are disproportionately expensive, not just disproportionately frequent.** 29.9% of orders carry 43.8% of refund dollars (\$4.64M); Fraudulent Return claims average 3.1× a legitimate return.
2. **The classifier earns its complexity.** Random Forest reaches perfect precision/recall on the binary "needs a second look" framing where a free 4-flag heuristic tops out at F1 0.61 (Section 5.1) — though the 0.999 multi-class F1 itself is a synthetic-data artifact, not a production expectation.
3. **The quality problem is broad-based, not one bad vendor.** The Section 5.2 deep-dive found quality-issue rates banded tightly (31.5%–35.2%) across all 12 product categories — Cluster 1's 38% quality-issue share is a systemic pattern hiding inside the "legitimate" majority, not a single vendor escalation waiting to be found.
4. **A simpler model wins honestly on refund-exposure sizing.** Linear Regression beats both the naive "refund = order value" baseline (40% RMSE reduction) and the more complex GBTR regressor (Section 5.3) — refund amount is close to linear in order value, and forcing a non-linear model onto a linear relationship doesn't help.
5. **The dataset's class prevalence is realistically calibrated, even if separability isn't.** Our combined Fraudulent Return + Wardrobing rate (17.9%) sits close to Apriss Retail's industry-reported 15.14% fraud rate (Section 1) — the scale of the problem this project targets is real, even though the classifier's near-perfect accuracy on this specific dataset is not one to expect on live data.

7 · Strategic Recommendations

1. **Require photo evidence above a value threshold — ship this before the model.** The "high-value item + no photo evidence" pattern alone touches **\$1.88M** of non-legitimate refund exposure (4,673 orders) and needs no ML: a policy rule, live immediately, independent of the classifier's rollout timeline.
2. **Deploy the fraud classifier as a triage score, not an auto-block.** Route the highest `fraud_signal_score` decile plus Cluster 0 members to manual fraud review; auto-approve the lowest-risk tail. Re-validate precision/recall on live traffic before trusting this dataset's 0.999 F1 — budget for 80–90% in the initial production SLA. Prioritize Fraudulent Return claims specifically: at \$434.73 average vs. \$141.14 for legitimate returns, each one caught is worth ~3x a typical legitimate claim in review effort saved.
3. **Give Cluster 1's 38% quality-issue rate to QA as a systemic review, not a vendor hunt.** Section 6's finding that quality-issue rates are broad-based (31.5%–35.2% across every category) means the fix is a general QA/packaging/description-accuracy audit, not a search for one bad vendor — Tools (35.2%) and Electronics (34.2%) are the highest-rate starting points if a sequenced rollout is needed.
4. **Tighten policy, don't investigate, for Cluster 2.** High order/return volume, low order value, Policy-Abuser-dominated — a shorter return window or restocking fee targets this segment more precisely than a fraud investigation would.
5. **Re-threshold before deployment, using the Section 5.1 sweep.** A false "Legitimate → Fraudulent Return" flag costs more in customer trust than a slow manual review costs in ops time — the precision/recall trade-off across thresholds 0.1–0.9 is already mapped; picking a production threshold needs real cost estimates, not a re-run of the analysis.
6. **Feed the refund-exposure regressor to Finance, not just Fraud & Risk.** Section 5.3's Linear Regression prices expected refund dollars the moment a return is filed — a reserve- accounting input independent of whether the return turns out to be fraud, abuse, or genuine.
7. **Re-validate quarterly.** Fraud patterns adapt to whatever the model learns; without a temporal holdout in this dataset, we cannot measure drift here — production deployment needs a rolling time-based validation window from day one.

Risks & future roadmap

- **Risk — synthetic-data overfit to production:** mitigated by treating this model as a pipeline proof-of-concept and re-training on real labeled fraud outcomes before go-live.
- **Risk — no ground-truth fraud labels in production:** real retailers rarely have a clean `abuse_type` column; roadmap includes a human-in-the-loop labeling process from the manual review queue this system creates.
- **Risk — broad-based quality findings need root-cause work we didn't do.** Section 6's quality deep-dive tells us the problem isn't one vendor, but not *why* defect rates are banded around 33% everywhere — that's a follow-up analysis (packaging audit, supplier QA sampling), not something this dataset can answer on its own.
- **Future:** real-time scoring at return-request time (not batch), a feedback loop from review-team overrides back into re-training, extending clustering features with unstructured support-ticket

text once the Flume/API ingestion (Section 3) is live, and retraining the refund- exposure regressor as a monitoring signal for reserve-accounting accuracy over time.

Appendix

A. AI usage disclosure

Claude (Anthropic) was used throughout this project as an AI pair-programmer and analyst: drafting the PySpark data-engineering and MLlib modeling notebooks, generating this report and the accompanying executive deck, and proposing the feature-engineering rationale in Section 4.3. All code was executed end-to-end against the real 60,000-row dataset (not fabricated output) and every metric quoted in this report was read directly from that execution — see `docs/fraud-classification-results.csv`, `docs/rf-feature-importance.csv`, `docs/quality-clustering-results.csv`, `docs/baseline-comparison-results.csv`, `docs/threshold-analysis-results.csv`, `docs/silhouette-selection-results.csv`, `docs/quality-deep-dive-results.csv`, `docs/regression-results.csv`, and `docs/regression-feature-importance.csv` for the underlying numbers. Industry statistics in Section 1 are drawn from published third-party research, cited in Appendix C, not generated by the model. Team members reviewed, validated, and take ownership of the business framing, architecture decisions, and final recommendations.

B. Team contributions

Name	Roll no.	Workstream	Artifacts owned
Subham Ranjan	A017	Data acquisition & Bronze/Silver pipeline	<code>notebooks/01_data_engineering.ipynb</code>
Shriya	A023	Feature engineering & Gold layer	<code>notebooks/01_data_engineering.ipynb</code> §4
Abhinav Kumar	A035	Classification modeling & evaluation	<code>notebooks/02_ml_modeling.ipynb</code> §2
Dev Vagreacha	A049	Clustering, quality deep-dive & regression	<code>notebooks/02_ml_modeling.ipynb</code> §3–4
Samruddhi Pradhan	A054	Architecture & enterprise design	Report §3
Yatharth Vij	A072	Narrative, report & deck	This report; presentation deck

C. References

- Dataset: `ecommerce_return_abuse_dataset.csv`, Kaggle.
- Apache Spark MLlib documentation — classification, regression, clustering, and feature transformers.
- Project brief: `docs/ProjectGuideLines_Big_Data_Analytics_Project_Guidelines.pdf`.
- 1. National Retail Federation, *2025 Retail Returns Landscape* — [nrf.com/research/2025-retail-returns-landscape](https://www.nrf.com/research/2025-retail-returns-landscape).

- 2. Appriss Retail (with Deloitte), *Fraudulent Returns and Claims Cost Retailers \$103B in 2024* — apprissretail.com.